

Lazy Loading vs Eager Loading in Singleton Design Pattern using C#

In this article, I will discuss **Lazy Loading vs Eager Loading in Singleton Design Patterns in C#** to Achieve Thread Safety with Examples. Please read our previous article discussing [How to Implement a Thread-Safe Singleton Design Pattern in C# using a Lock and Double-Check Locking](#) mechanism with Examples. As part of this article, we will discuss the following pointers.

1. **What is Non-Lazy or Eager Loading?**
2. **What is Lazy or Deferred Loading?**
3. **Understanding The Lazy<T> Generic Class in C#.**
4. **How can the Singleton Design Pattern Thread-Safe be made using Eager Loading and Lazy Loading in C#?**
5. **When to use Lazy or Eager Loading in Singleton Design Pattern?**

Note: The Singleton design pattern ensures that a class has only one instance and provides a global access point to that instance. Lazy and Eager loading are two different approaches to initializing an instance in the Singleton Design Pattern in C# (or any programming language).

What is Non-Lazy or Eager Loading in C#?

Eager loading creates the Singleton instance when the class is loaded or before it's needed. This ensures that the instance is available immediately when requested, but it may consume resources even if it's never used during the application's lifetime.

So, the Eager Loading or Non-Lazy Loading in C# is nothing but a process in which we need to initialize the Singleton Object at the time of application start-up rather than on-demand and keep it ready in memory for future use. The advantage of using Eager Loading in the Singleton Design Pattern is that the CLR (Common Language Runtime) will take care of Object Initialization and Thread Safety in Multithread Environment. That means we will not be required to write any code explicitly for handling the thread safety for a multithreaded environment.

Example to Understand Non-Lazy or Eager Loading in Singleton Design Pattern:

Let's understand How to Implement the Singleton Design Pattern using Eager Loading in C# with one Example. So, create a class with the name **Singleton.cs** and copy and paste the following code. In the example below, while creating the singletonInstance variable, we are also initializing that variable and making it ready to be used. This is nothing but Eager Loading of Singleton Instance. Now, the CLR will take care of object initialization and Thread-Safety in a Multithreaded Environment. Within the GetInstance() method, we don't need to write the object initialization, null checking, and thread-safety code, as the CLR will handle all these things. As part of the GetInstance() method, we must return the Singleton Instance. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace SingletonDemo
{
    public sealed class Singleton
    {
        //This variable value will be increment by 1 each time the object
        of the class is created
        private static int Counter = 0;
        //This variable is going to store the Singleton Instance
        //Initiazling the Variable at the time of class start-up and make
        it ready to be used in Future
        private static readonly Singleton singletonInstance = new Singleton();
    }
}
```

```

        //The following Static Method is going to return the Singleton
Instance
        //This is Thread-Safe as it use Eager Loading
        public static Singleton GetInstance()
        {
            //Now we don't need to use any NULL Checking code
            //We don't need to write Explicit code for thread-safety
            //Object initialization and Thread-Safety will be taken care by
CLR as we are using Eager Loading
            //Return the Singleton Instance
            return singleInstance;
        }
        private Singleton()
        {
            //Each Time the Constructor is called, increment the Counter
value by 1
            Counter++;
            Console.WriteLine("Counter Value " + Counter.ToString());
        }
        //The following method can be accessed from outside of the class by
using the Singleton Instance
        public void PrintDetails(string message)
        {
            Console.WriteLine(message);
        }
    }
}

```

Next, modify the Main method of the Program Class as shown below. Here, you can see that using Parallel Programming. We are accessing the GetInstance() method of the Singleton class using two different threads, i.e., accessing the Singleton Instance in a multithread environment. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.Threading.Tasks;
namespace SingletonDemo
{
    class Program
    {
        //Example to Understand Thraed-Safe Singleton Design Pattern using
Eager Loading
        //When use in a Multithreaded Environment
        static void Main(string[] args)
        {
            //The following Code will Invoke both methods Parallely using
two different Threads
            Parallel.Invoke(
                //Let us Assume PrintTeacherDetails method is Invoked by
Thread-1
                () => PrintTeacherDetails(),
                //Let us Assume PrintStudentdetails method is Invoked by
Thread-2
                () => PrintStudentdetails()
            );
            Console.ReadLine();
        }
    }
}

```

```

    }
    private static void PrintTeacherDetails()
    {
        //Thread-1 Calling the GetInstance() Method of the Singleton
class
        Singleton fromTeacher = Singleton.GetInstance();
        fromTeacher.PrintDetails("From Teacher");
    }
    private static void PrintStudentdetails()
    {
        //At the same time, Thread-2 also Calling the GetInstance()
Method of the Singleton Class
        Singleton fromStudent = Singleton.GetInstance();
        fromStudent.PrintDetails("From Student");
    }
}
}

```

Now run the application, and you will see the following output.

```

Counter Value 1
From Teacher
From Student

```

You can see that only one instance was created as the Counter value is 1. The above Singleton Class is thread-safe after removing the lock variables and null checking. The CLR (Common Language Runtime) will internally take care of the variable initialization and thread safety in the multithreaded environment as we use Eager Loading.

What is Lazy Loading or Deferred Loading in C#?

As the name suggests, lazy loading defers the creation of the Singleton instance until it is needed. This can be beneficial if the initialization of the instance is expensive or if you want to save resources by not creating the instance until it's necessary.

Lazy Loading or Deferred Loading in C# is a concept commonly used to delay the initialization of an object until the point at which it is needed. So the main objective of Lazy Loading or Deferred Loading is to load the object on-demand or, you can say, the object when needed. The most important point you need to remember is that you need to use Lazy loading when the cost of the object creation is very high, and the use of that object is rare. Lazy loading improves the performance of an application if it is used properly.

Example to Understand Lazy or Deferred Loading in Singleton Design Pattern:

We have already discussed Lazy Loading or Deferred Loading in Singleton Design Patterns. In our previous article, we discussed How to Implement a Thread-Safe Singleton Design Pattern using a Lock and Double Check Locking Mechanism. These two use the Lazy Loading Mechanism, i.e., the Singleton Instance is created when we access the object for the first time. The second time onwards, it uses the already created instance. As developers, we need to write the code to make the Singleton Instance thread-safe.

With C# 4.0 onwards, we can use the Lazy<T> Generic class to make the Singleton Instance Lazy Loading. Let us proceed and see how we can implement Lazy Loading in the Singleton Design Pattern using the Lazy<T> Generic Class in C#.

Example to Understand Lazy Loading of Singleton Instance using Lazy<T> Generic Class.

The Lazy<T> Generic Class, introduced as part of .NET Framework 4.0, provides built-in support for lazy initialization, i.e., on-demand object initialization. If you want to make an object (such as Singleton) lazily initialized, i.e., on-demand object initialization. You need to pass the type (Singleton) of the object to the Lazy generic class.

Within the Lazy<T> constructor, using the lambda expression, we need to create the instance, which is shown below.

```
Lazy<Singleton> SingletonInstance = new Lazy<Singleton>(() => new Singleton());
```

The most important point you must remember is that the Lazy<T> objects are, by default, thread-safe. In a multi-threaded environment, the Lazy object will ensure thread safety when multiple threads try to access the same GetInstance Method simultaneously. So, to use the Lazy Loading of the Singleton instance using the Lazy<T> generic class, please modify the Singleton class as follows. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
namespace SingletonDemo
{
    public sealed class Singleton
    {
        //This variable value will be increment by 1 each time the object
        of the class is created
        private static int Counter = 0;
        //This will make the Singleton Instance Lazy Loading
        //Lazy Object are Thread-Safe by default
        private static readonly Lazy<Singleton> SingletonInstance = new
        Lazy<Singleton>(() => new Singleton());
        //The following Static Method is going to return the Singleton
        Instance
        //This is Thread-Safe as it uses Lazy<T> Object
        public static Singleton GetInstance()
        {
            //Now we don't need to use any NULL Checking code
            //We don't need to write Explicit code for thread-safety
            //Call the Value property of the Lazy object which will return
            the Singleton Instance
            return SingletonInstance.Value;
        }
        private Singleton()
        {
            //Each Time the Constructor is called, increment the Counter
            value by 1
            Counter++;
            Console.WriteLine("Counter Value " + Counter.ToString());
        }
        //The following method can be accessed from outside of the class by
        using the Singleton Instance
        public void PrintDetails(string message)
        {
            Console.WriteLine(message);
        }
    }
}
```

With the above changes, now run the application and see the output.

```
Counter Value 1
From Teacher
From Student
```

We got the same output as Eager Loading. This is because the lazy keyword creates only one singleton instance, and they are, by default, thread-safe. This is why we do not get any error while invoking the methods parallelly.

When to Lazy Loading vs. Eager Loading in Singleton Design Pattern using C#?

The choice between lazy and eager loading in the Singleton design pattern depends on your application's requirements and constraints.

Use lazy loading when:

- **Resource Optimization:** If the Singleton instance is heavy and consumes significant resources, lazy loading ensures that these resources are not utilized until absolutely necessary. This is useful if there's a chance that the instance might not be needed at all during the application's runtime.
- **Start-up Performance:** In applications where start-up time is crucial, lazy loading can help reduce the start-up load by deferring the creation of heavy objects to a later point. This is common in desktop applications where initial responsiveness is critical.
- **Conditional Initialization:** If your application has multiple execution paths and the Singleton is not required for every path, lazy loading can prevent unnecessary initialization, thus saving resources.

Use eager loading when:

- **Predictability:** Eager loading contributes to the predictable behavior of the application by initializing the Singleton instance during application startup. Since initialization sequences are consistent, this can simplify debugging and behavior analysis.
- **Concurrency Simplification:** In multi-threaded applications, eager loading can simplify the design by avoiding the need for synchronization mechanisms required for safely lazy loading the instance. Once the instance is created during startup, it can be accessed by multiple threads without additional overhead or complexity.
- **Performance Critical Situations:** If the Singleton is used in performance-critical parts of the application and must be accessed quickly without delay, having it already created and available (eager loading) ensures that there is no delay in accessing the instance when needed.

In the next article, I will discuss the [Difference Between Static Class and Singleton Class in C#](#) with Examples. In this article, I explain **Lazy Loading vs. Eager Loading in Singleton Design Patterns using C#** with Examples. I hope this C# Lazy Loading and Eager Loading in the Singleton Design Pattern article will help you with your needs. I would like your feedback. Please post your feedback, questions, or comments about this Lazy Loading vs. Eager Loading in the Singleton Design Pattern article.